

An Expectiminimax-Based Texas Hold'em Agent with Sampled Chance Nodes: Design, Implementation, and Empirical Analysis of Decision Quality

Nadira Halim Himla 2221920642
Sajiah Islam Chowdhury 2212447642
S. Anam Ridwan Shanto 2022563042
Mohammad Robiul Islam Hasib 1911677642
Isnat Hossain Rijon 2211203042
Group Number-7 Section:1

Department of Electrical and Computer Engineering
North South University, Dhaka, Bangladesh
CSE 440 — Artificial Intelligence

Abstract—Poker is a stochastic game of imperfect information in which an agent must commit to a betting action before future card deals are revealed and without observing its opponent's private cards. This paper presents the design, implementation and empirical analysis of a two-to-four player Texas Hold'em system whose artificial player selects actions by depth-limited Expectiminimax search over a game tree containing MAX, MIN, chance and heuristically evaluated leaf nodes. Because exhaustive expansion of the remaining deck is costly, chance nodes are expanded by uniform sampling without replacement with a configurable branching factor, yielding an unbiased estimate of the expected utility. The system is written in Python with a Tkinter interface and additionally implements a Monte Carlo win-probability simulator, a phase-dependent hybrid controller, side-pot and tournament logic, and a regression test suite. Beyond reporting headline win rates, we analyse the agent itself. Across 200-hand heads-up matches it wins 99.5% of hands against a random opponent but 50.5% and 54.0% against rule-based and always-call opponents, and raising the search depth from one to five plies increases mean decision time from 0.027 ms to 4.554 ms with no statistically distinguishable change in win rate. Both observations follow from the evaluation function: its output is bounded below by a positive constant while a fold is scored as the negative of the pot, so folding is a dominated action and was never selected in 1600 sampled decision states. We formalise this property, quantify a 44.3% disagreement between the two hand evaluators present in the system, and establish the minimum depth at which a chance node becomes reachable.

Index Terms—Adversarial search, Expectiminimax, chance nodes, imperfect-information games, computer poker, Monte Carlo simulation, heuristic evaluation, decision making under uncertainty.

I. INTRODUCTION

Adversarial search was developed for deterministic games of perfect information, in which both players observe the complete state and every legal move yields a single successor. Poker satisfies neither assumption. Cards are dealt from a

shuffled deck, so an action taken on the flop is followed by a random transition into one of many possible turn states; and each player's hole cards are private, so no participant ever observes the true game state. These two properties make poker a long-standing benchmark for research on decision making under uncertainty [4], [3], [5].

Minimax cannot be applied unmodified, because it assumes that the value of an internal node is determined entirely by the two players' choices. Expectiminimax removes that assumption by introducing a third node type whose value is the probability-weighted average of its children [1]; Ballard's *-minimax family later showed how cutoffs can be propagated through such nodes by bounding the achievable weighted average [2]. Contemporary competitive agents take a different route entirely, approximating Nash equilibria through counterfactual regret minimisation and its descendants [8], [9], [10], [11], [12]. Those methods are outside the scope of an undergraduate implementation project, but Expectiminimax remains a suitable vehicle for studying how stochastic transitions, search depth and heuristic evaluation interact, because every term in its value computation is directly inspectable.

This paper describes such a system and then subjects it to measurement. The motivation for the second emphasis is that a search algorithm is only as good as the leaf evaluation it backs up, and an implemented agent can appear competent for reasons unrelated to its search. We therefore treat the source code as the object of study and report what the system actually does. The contributions are:

- 1) a complete Texas Hold'em engine with best-of-seven hand evaluation, blinds, all-in handling, layered side pots, tournament progression and a graphical interface;
- 2) a depth-limited Expectiminimax agent whose chance nodes are expanded by uniform sampling without re-

placement, with the branching factor exposed as a parameter, together with a phase-dependent hybrid controller and a Monte Carlo win-probability simulator;

- 3) a structural result showing that, under the implemented transition rule, a chance node is unreachable at depth limits below three, so the two default difficulty presets execute deterministic minimax rather than Expectiminimax;
- 4) a proof, with empirical confirmation, that folding is dominated under the implemented evaluation function: 1340 raises, 260 calls and zero folds across 1600 sampled decision states; and
- 5) a reproducible measurement harness and results covering search cost against depth and branching factor, win rate against three baselines, simulator convergence, and cross-validation of the two hand evaluators present in the system.

II. RELATED WORK

Expectiminimax originates in the treatment of stochastic events as expectation nodes within an otherwise adversarial tree, and is the standard textbook formulation for games of chance [1]. Ballard generalised the procedure to *-minimax, deriving conditions under which a chance node can be cut off before all of its children have been evaluated [2]. Neither the present system nor the classical formulation prunes: the search implemented here is full-width, a point returned to in Section IX.

Koller and Pfeffer established the game-theoretic framing of imperfect-information problems and the sequence-form representation that made large extensive games computationally tractable [3]. Within poker specifically, Billings *et al.* surveyed the obstacles the domain poses to search-based agents and described Poki, which combined hand-strength computation, opponent modelling and simulation [4]. Their earlier work established selective sampling simulation as a practical substitute for exhaustive expansion, showing that repeatedly playing a position out to showdown against sampled opponent holdings yields a usable estimate of expected value [6]. The chance-node treatment adopted in this work is related in spirit, though considerably simpler: future cards are sampled and the resulting values averaged, rather than reasoned about as an information set. Southey *et al.* addressed the complementary problem of modelling the opponent rather than the cards, inferring a posterior over opponent strategies and responding to that distribution [7]; the present system collects the frequency statistics such a model would require but does not use them, as Section VI notes. Rubin and Watson provide a broad review of the field and of the methodological transition away from heuristic search [5].

That transition is now essentially complete. Counterfactual regret minimisation [8] enabled heads-up limit hold'em to be solved [9], and subsequent systems combined regret minimisation with continual re-solving, deep networks and depth-limited subgame search to defeat professional players at no-limit hold'em in both the two-player [10], [11] and

multiplayer [12] settings. A parallel line of work replaces heuristic leaf evaluation with statistical sampling inside the tree itself, through Monte Carlo tree search and the UCT selection rule [13], [14], which is the natural successor to the sampled chance node used here. The present work makes no claim to compete with these systems; it is an implementation and empirical study of the classical formulation, and the comparison is used only to situate the design.

III. PROBLEM FORMULATION

A single hand is modelled as a finite extensive-form process. A state s comprises the agent's private cards H , the public board B , the undealt deck D , the pot Π , the amount owed by the acting player β , the phase $\varphi \in \{\text{preflop}, \text{flop}, \text{turn}, \text{river}, \text{showdown}\}$, the mover label $\tau \in \{\text{MAX}, \text{MIN}, \text{CHANCE}\}$, the number of actions already taken in the current betting round κ , and the acting player's stack χ . All card sets are subsets of the 52-card deck $\mathcal{C}$, where $|\mathcal{C}| = 4 \times 13 = 52$.

The action set at a decision node is

$$A(s) = \begin{cases} \{\text{fold}, \text{check}, \text{raise}\}, & \beta \leq 0, \\ \{\text{fold}, \text{call}, \text{raise}\}, & \beta > 0, \end{cases} \quad (1)$$

with *raise* omitted when the player cannot cover it. Transitions are deterministic at MAX and MIN nodes and stochastic at CHANCE nodes, where the next public card o_i is drawn from D . A hand terminates when a player folds, awarding the pot to the survivor, or at showdown, where the pot is split among the players holding the strongest five-card combination. The agent selects

$$a^* = \arg \max_{a \in A(s)} V(T(s, a)), \quad (2)$$

where T is the transition function and V the backed-up value.

Two properties separate this from deterministic board-game search. First, the branching factor at a chance node equals $|D|$, which is 48 immediately after the hole cards are dealt in a heads-up hand, so a chance ply is an order of magnitude wider than a betting ply. Second, the opponent's cards are never observed, so the true node value is defined over an information set rather than a state [3]. The implemented agent addresses the first by sampling and the second by substitution, evaluating every node—including MIN nodes—using its own hand.

IV. SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the architecture recovered from the source. A Tkinter presentation layer drives a game engine that owns the authoritative state and delegates each AI decision to a strategy selector. The selector dispatches to either the Expectiminimax searcher or a cheaper single-shot heuristic; both paths use the same evaluation function. The engine occupies `game_engine/`, the decision logic occupies `algorithm/`, and the interface occupies `frontend/`. Their principal modules implement cards, hand evaluation, game and tournament state, legal actions, tree construction, search, simulation, strategy selection, and the user interface.

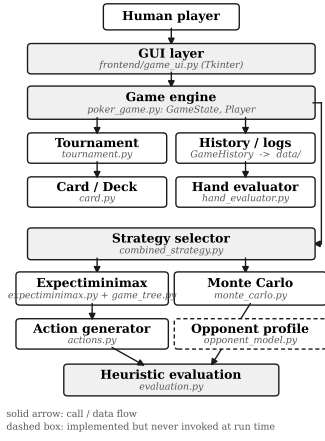


Fig. 1. Overall architecture of the implemented poker system, recovered from the import graph of the source tree. Arrows denote call or data flow. The dashed box marks a module that is implemented and tested but never invoked at run time.

The engine advances a hand through the sequence preflop, flop, turn, river, showdown. A hand opens with blind posting and two private cards per player followed by a preflop betting round; each subsequent phase burns one card, deals the flop (three cards), the turn or the river (one card each), and runs a further betting round. If all but one player folds, the hand short-circuits to pot award.

A notable design characteristic is that the engine and AI subsystem use different card representations. The engine manipulates `Card` objects, whereas the AI receives a plain dictionary containing a snapshot of the state. This single conversion boundary keeps the search free of engine dependencies, but it is also why the system contains two independent hand evaluators, a point quantified in Section VII.

V. METHODOLOGY

A. Game State Representation

The state passed to the search is a flat dictionary whose keys map one-to-one onto the symbols above: `player_hand` (H), `community_cards` (B), `deck` (D), `pot` (Π), `current_bet` (β), `phase` (φ), `to_move` (τ), `actions_this_phase` (κ) and `player_chips` (χ). Successors are produced by shallow copying, so the deck list is shared between siblings except at chance nodes, where a filtered copy is created. Legal actions follow (1) and successors are expanded in the fixed order *raise*, *call*, *check*, *fold*; because the search is full-width and unpruned this ordering does not affect the value returned, only which action is reported when two tie.

B. Game-Tree Construction and MAX, MIN and CHANCE Nodes

Tree construction is recursive over three node types. A node is a leaf when the depth budget is exhausted or the phase is terminal; otherwise its type is given by τ . After an action is applied the mover alternates while $\kappa < 2$; once two actions have been taken in a betting round the state becomes a chance

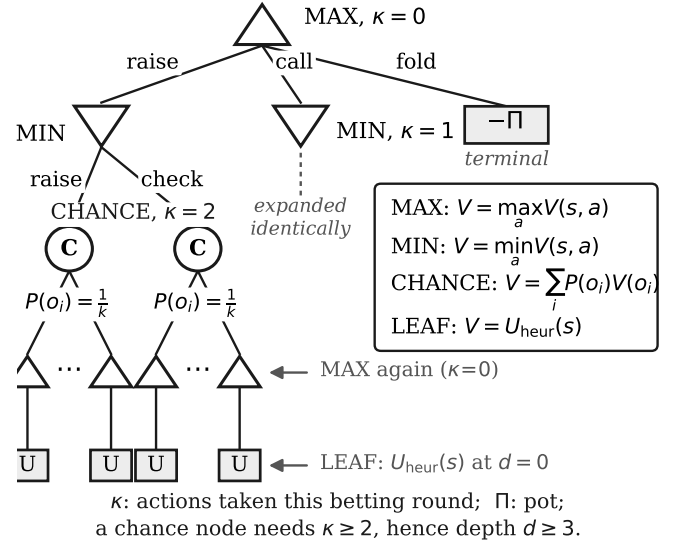


Fig. 2. Expectiminimax search tree as generated by the implementation, showing decision, adversarial and stochastic states. Fold branches are terminal and scored directly from the pot; a chance node receives one child per sampled card with uniform probability $1/k$; leaves are scored by the heuristic evaluation function.

node, or a showdown if the phase is already the river. A chance node resets κ to zero and returns the move to MAX. Fig. 2 illustrates the resulting structure.

This rule has a consequence that is central to interpreting the experiments. Search always begins from a freshly constructed state with $\kappa = 0$, so two decision plies must be expanded before a chance node can appear:

$$d \geq 3 \text{ is necessary for any chance node to be expanded.} \quad (3)$$

At $d \in \{1, 2\}$ the algorithm degenerates to plain minimax over a deterministic tree and the expectation machinery is never exercised.

C. Expectiminimax

The value of a node is defined by its type:

$$V(s) = \max_{a \in A(s)} V(T(s, a)), \quad \tau = \text{MAX}, \quad (4)$$

$$V(s) = \min_{a \in A(s)} V(T(s, a)), \quad \tau = \text{MIN}, \quad (5)$$

$$V(s) = \sum_{i=1}^k P(o_i) V(o_i), \quad \tau = \text{CHANCE}, \quad (6)$$

$$V(s) = U_{\text{heur}}(s), \quad \text{leaf.} \quad (7)$$

Here (4) expresses the agent's objective of maximising its own utility, (5) the adversarial choice attributed to the opponent, and (6) the expected utility over the stochastic card deal. Terminal states are scored directly: an opponent fold yields $+\Pi$ and an agent fold $-\Pi$. Algorithm 1 states the procedure as implemented.

Construction is separated from backup: the tree is materialised as nested dictionaries and then traversed, which is

Algorithm 1 Expectiminimax search as implemented

```

1: Input: state  $s$ , depth  $d$ , sample size  $k$ 
2: Output: best action  $a^*$  and value  $V$ 
3: function BUILD( $s, d$ )
4: if  $d \leq 0$  or  $\varphi(s)$  is terminal then
5:   return leaf node holding  $s$ 
6: else if  $\tau(s) = \text{CHANCE}$  then
7:   sample  $k$  cards from  $D$  without replacement
8:   attach BUILD( $o, d - 1$ ) with  $P = 1/k$  for each sample
9:   return chance node
10: else
11:   attach BUILD( $T(s, a), d - 1$ ) labelled  $a$ , for  $a \in A(s)$ 
12:   return node of type  $\tau(s)$ 
13: end if
14: function BACKUP( $n$ )
15: if  $n$  is a leaf then
16:   return  $U_{\text{heur}}(s_n)$ 
17: else if  $n$  is a chance node then
18:   return  $\sum_{c \in n} P(c) \text{BACKUP}(c)$ 
19: else
20:   return max (at MAX) or min (at MIN) over children
21: end if
22:  $(a^*, V) \leftarrow \text{BACKUP}(\text{BUILD}(s, d))$ 
23: if SHOULDBLUFF(strength( $s$ ),  $\Pi$ ) and raise is legal then
24:    $a^* \leftarrow \text{raise} \{ \text{post-hoc override} \}$ 
25: end if
26: return  $(a^*, V)$ 

```

memory-inefficient but makes the node structure inspectable. No pruning of any kind is applied, so the search is full-width. One deviation from the textbook procedure should be recorded: after the search returns, a stochastic bluffing rule may overwrite the chosen action with a raise, firing with probability 0.15 when raw hand strength is at most 2 on a 0–9 scale and the pot is at least 100 chips. Because it acts on the search output rather than inside the value computation, it is a policy-level perturbation and not part of the expected-utility optimisation.

D. Chance-Node Sampling

Exhaustive expansion creates one child per remaining card. With $|D| = 48$ and three actions per betting ply, a depth-five tree holds on the order of $3^2 \cdot 48 \cdot 3^2$ nodes for a single chance ply, and a second chance ply multiplies that by a further $|D| - 1$. The implementation therefore draws

$$k = \min(k_{\max}, |D|) \quad (8)$$

distinct cards uniformly without replacement and assigns each outcome

$$P(o_i) = \frac{1}{k}, \quad i = 1, \dots, k. \quad (9)$$

Since both the sampling and the true distribution over the next card are uniform, (6) is an unbiased estimate of the expectation over D , with variance decreasing in k . The parameter

supplying $k_{\max}$ defaults to 200, truncated to $|D|$ by (8), and is overridden by the difficulty presets with 5 or 10.

E. Heuristic Evaluation

Leaf states are scored by an additive function of five components, each computed directly from the state and clipped to a fixed range. Writing $\rho \in [0, 9]$ for the raw hand-category score and $\delta(B)$ for a board-danger penalty,

$$h(s) = \min(40, \frac{40}{9}\rho), \quad (10)$$

$$p(s) = \begin{cases} 10, & \beta \leq 0 \\ \min(20, 4\Pi/\beta), & \beta > 0 \end{cases} \quad (11)$$

$$q(s) = \begin{cases} 5, & \kappa = 0 \\ 15, & \kappa > 0 \end{cases} \quad (12)$$

$$b(s) = \begin{cases} 15, & |B| < 3 \\ \max(0, 15 - \delta(B)), & |B| \geq 3 \end{cases} \quad (13)$$

$$c(s) = \begin{cases} 10, & \Pi \leq 0 \\ \min(10, 2\chi/\Pi), & \Pi > 0, \end{cases} \quad (14)$$

and the leaf value is the unweighted sum

$$U_{\text{heur}}(s) = h(s) + p(s) + q(s) + b(s) + c(s). \quad (15)$$

Here h measures hand-category strength, p pot odds, q position within the betting round, b board texture, and c the stack-to-pot ratio; $\delta(B)$ adds 8 when three or more board cards share a suit and 7 when three or more form a consecutive run, so a coordinated board lowers the score. No weight vector is learned or tuned: relative importance is encoded implicitly in the clip ranges, which give $U_{\text{heur}}(s) \in [5, 100]$. That lower bound determines much of the agent’s observed behaviour.

Proposition 1 (fold domination). *For any non-terminal state s with $\Pi \geq 0$, a successor reached by folding is never strictly preferred at a MAX node.*

Proof. An agent fold produces a terminal state scored $-\Pi \leq 0$. Every non-folding successor is either terminal with value $+\Pi \geq 0$, or is backed up from leaves satisfying $U_{\text{heur}} \geq 5$ by (10)–(15), since every component is non-negative and $q(s) \geq 5$. A weighted average of values bounded below by 5 is itself bounded below by 5, and a maximum or minimum over such values is likewise at least 5. Hence every non-folding successor has value at least $0 \geq -\Pi$, with strict inequality whenever $\Pi > 0$. ■

The proposition does not assert that the agent is badly designed; it states that under this leaf function, folding can never be selected by (4), so the agent’s fold frequency is a property of the evaluation function rather than of the search. Section VII confirms this empirically.

F. Monte Carlo Probabilistic Simulation

Two distinct Monte Carlo procedures are implemented. The first completes the board from a shuffled copy of the deck, scores the agent’s hand and averages over N repetitions to obtain a mean strength $\bar{\rho}$; a policy wrapper raises when

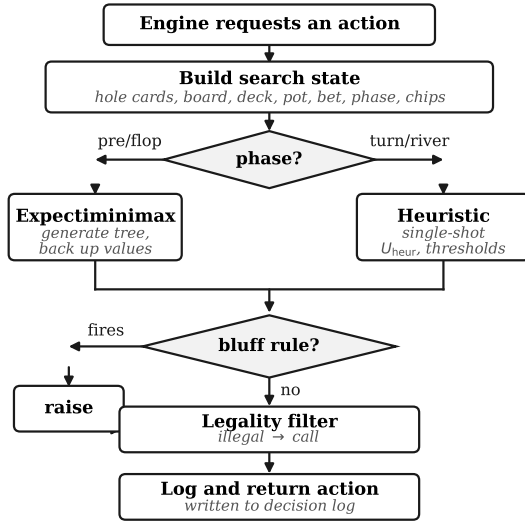


Fig. 3. AI decision-making flow. The search state is rebuilt for every request; the phase selects the expensive or cheap path; the bluffing rule and the engine’s legality filter are applied after the strategy returns.

$\bar{\rho} \geq 5$, calls or checks when $\bar{\rho} \geq 2$ or nothing is owed, and otherwise folds. Because the hand-category scale is heavily skewed towards low values this threshold is conservative, as Section VII shows.

The second is a showdown simulator in the tradition of selective sampling simulation [6]: it deals hole cards to each opponent, completes the board, computes the best five-card hand for every player, and accumulates a win with weight 1 and a tie with weight 0.5, giving

$$\hat{P}_{\text{win}} = \frac{N_{\text{win}} + \frac{1}{2}N_{\text{tie}}}{N}. \quad (16)$$

The interface converts this estimate into a five-point star rating for the human player. It is used only for display and does not feed the agent’s decision.

G. Hybrid Strategy Selection

The hybrid controller selects a procedure by betting phase, as shown in Fig. 3: Expectiminimax search on the preflop and flop, and a single-shot heuristic on the turn and river. The cheap path evaluates $U_{\text{heur}}(s)$ once, raising above 65, calling or checking above 30 or when nothing is owed, and otherwise folding. The bluffing override applies on this path as well. The motivation is computational, since later streets contribute the bulk of the decisions in a completed hand; Section VII quantifies the saving. The thresholds 65 and 30 are fixed constants in the source and were not derived from any documented tuning procedure.

VI. IMPLEMENTATION

The system is implemented in Python 3 in roughly 2400 lines across 25 modules. The only third-party dependencies are NumPy, Matplotlib and pytest; the interface uses Tkinter,

with Matplotlib embedded through the TkAgg backend for the chip-count statistics window.

Cards and evaluation. Ranks are integers 2–14 and suits are strings, giving the standard 52-card deck. Showdown evaluation enumerates all $\binom{7}{5} = 21$ five-card subsets of a player’s seven cards and keeps the maximum under a lexicographic ordering of (category, tie-breaker list). This is the correct procedure for Texas Hold’em because the best hand may use any combination of hole and board cards. Wheel straights are handled by demoting the ace in the tie-breaker list.

Betting and pots. Each player tracks a per-round and a per-hand contribution. Calls and raises are clamped to the available stack, and a player reaching zero chips is marked all-in. At showdown, side pots are rebuilt from the distinct per-hand contribution levels: each layer amount is the increment multiplied by the number of players who contributed at least that much, with eligibility restricted to non-folded contributors.

Interface, tournament, logging and testing. The interface renders cards on Tkinter canvases, exposes fold, call and raise controls, offers three difficulty presets, shows a live decision log, and displays the human player’s win probability as a star rating from (16). Tournament mode reuses the engine, eliminating players whose stacks reach zero and multiplying the blinds by 1.5 every five rounds. Decisions, engine actions and completed hands are written to three log files. Fifteen tests cover dealing, hand evaluation, state reset, three- and four-player games, a tournament run to completion, an all-in side-pot case, and two integration tests including chip conservation across five consecutive hands; all pass in 0.07 s.

Known structural issues. Three affect how the results should be read. First, the system contains two independent hand evaluators: the engine uses the best-of-seven evaluator above, while the AI path uses a separate one in `algorithm/evaluation.py` that scores the first five known cards. They disagree in most positions, as quantified in Section VII. Second, the transition used inside the search sets the amount owed to zero after a raise, so the opponent at the following MIN node faces no bet and is offered *check* rather than *call*; the search therefore never models the opponent paying for the agent’s raise. Third, `algorithm/opponent_model.py` implements an `OpponentProfile` class that accumulates fold, call and raise frequencies and derives aggressiveness and bluff-likelihood statistics—the raw material a Bayesian opponent model would consume [7]—but no module in the game or decision path imports it.

VII. EXPERIMENTAL SETUP AND RESULTS

A. Setup

All measurements were produced by a seeded harness that calls the system’s own modules; no behaviour was re-implemented for the experiments. Runs used a single-core Intel Xeon at 2.10 GHz under Linux with CPython 3.12.3, and decision states were constructed exactly as `ai_make_decision` constructs them, so the search sees the same dictionary it would see during play.

TABLE I
SEARCH COST AS A FUNCTION OF DEPTH LIMIT ($k_{\max} = 200$,
TRUNCATED TO $|D| = 48$; 30 STATES PER ROW)

d	Mean nodes	Mean leaves	Time (ms)	Chance node reached
1	4	3	0.027 ± 0.008	no
2	10	7	0.051 ± 0.015	no
3	202	195	2.090 ± 0.175	yes
4	778	579	2.046 ± 0.743	yes
5	1930	1347	4.554 ± 1.156	yes

TABLE II
EFFECT OF THE CHANCE BRANCHING FACTOR AT $d = 4$ (20 STATES PER
ROW)

k	Mean nodes	Time (ms)	Fraction choosing raise
1	26	0.076	0.70
2	42	0.087	0.50
5	90	0.184	0.70
10	170	0.342	0.65
20	330	0.719	0.75
48	778	1.803	0.75

E1, cost versus depth. For $d \in \{1, \dots, 5\}$, 30 seeded preflop states were searched; node counts were obtained by traversing the materialised tree and timings by wall-clock measurement at the search entry point. *E2, branching factor.* With $d = 4$ fixed, $k_{\max}$ was varied over $\{1, 2, 5, 10, 20, 48\}$ across 20 seeded states. *E3, presets and phase cost.* The three shipped presets were measured as configured, and per-phase cost at $d = 4$ over 30 states per phase. *E4, match play.* Three agents—pure Expectiminimax at the default depth of four, pure Monte Carlo, and the hybrid—each played 200 independent hands against three baselines: a uniformly random legal action; a rule-based opponent that raises half the time when nothing is owed and folds half the time when facing more than 100 chips; and an always-call opponent. Both players were recreated with 1000 chips each hand and seats were alternated so that blind position could not bias the outcome. *E5, depth versus outcome.* The Expectiminimax agent played 200 hands at each $d \in \{1, \dots, 5\}$ against the always-call and rule-based baselines. *E6, estimator quality.* Convergence of (16) was measured from five independent estimates of the same state for each $N \in \{50, 100, 200, 500, 1000\}$ over 25 states; separately, the two hand evaluators were compared on 20 000 random seven-card sets.

For $n = 200$ hands the binomial standard error near parity is $\sqrt{0.25/200} \approx 3.5$ percentage points, giving a 95% confidence interval of about ± 6.9 points. All comparisons are read against that resolution.

B. Search Cost

Table I shows the cost of search. The jump from $d = 2$ to $d = 3$ is a factor of twenty in nodes and forty in time, against a factor of 2.5 from $d = 1$ to $d = 2$. This discontinuity is the empirical signature of (3): the first chance node appears at $d = 3$ and immediately multiplies the frontier by $|D| = 48$.

TABLE III
HEADS-UP MATCH RESULTS, 200 HANDS PER ROW, SEATS ALTERNATED.
THE 95% CONFIDENCE INTERVAL ON A WIN RATE IS ABOUT ± 6.9 POINTS

Agent	Opponent	Win (%)	Chips/hand	Time (ms)
Expectiminimax	Random	99.5	+58.15	2.349
Expectiminimax	Rule-based	50.5	−7.20	2.187
Expectiminimax	Always-call	54.0	+6.80	2.250
Monte Carlo	Random	24.0	−6.70	2.425
Monte Carlo	Rule-based	5.5	−11.60	2.402
Monte Carlo	Always-call	24.0	−5.30	2.428
Hybrid	Random	98.0	+49.75	1.992
Hybrid	Rule-based	42.5	−15.50	1.237
Hybrid	Always-call	55.0	+3.90	1.343

TABLE IV
WIN RATE AND COST AGAINST SEARCH DEPTH, 200 HANDS PER CELL

Opponent	Depth d				
	1	2	3	4	5
Always-call, win rate (%)	52.0	52.0	44.0	43.5	43.5
Always-call, net chips/hand	+5.0	+4.3	−9.1	−11.0	−11.0
Rule-based, win rate (%)	52.0	54.0	48.5	50.0	55.0
Rule-based, net chips/hand	+9.0	+14.8	−7.3	−0.4	+13.0
Mean decision time (ms)	0.07	0.08	0.93	2.00	4.48

Its absence at $d \leq 2$ confirms that the two lower difficulty presets run deterministic minimax.

Table II isolates the sampling trade-off. Node count and time grow essentially linearly in k , as expected for one sampled chance ply, and exhaustive expansion at $k = 48$ costs 1.803 ms against 0.342 ms at $k = 10$. The fraction of states selecting *raise* varies between 0.50 and 0.75 with no monotone trend, so at this depth the action choice is not strongly sensitive to the fidelity of the chance estimate.

The shipped presets give 4 nodes in 0.010 ms (easy, $d = 1$, $k_{\max} = 5$), 10 nodes in 0.019 ms (medium, $d = 2$, $k_{\max} = 10$) and 170 nodes in 0.337 ms (hard, $d = 4$, $k_{\max} = 10$). Per-phase costs at $d = 4$ were 1.836 ms preflop, 2.986 ms flop, 3.236 ms turn and 0.050 ms river; the river is cheap because two actions there transition directly to the terminal showdown, truncating the tree.

C. Match Play and Search Depth

Table III reports match play. The Expectiminimax agent wins 99.5% of hands against the random baseline, but that baseline folds with probability roughly 1/3 at every decision, forfeiting the pot outright. Against opponents that do not fold gratuitously the margin disappears: 50.5% against rule-based and 54.0% against always-call, both within one confidence interval of parity, with chip outcomes likewise indistinguishable from zero at this sample size. The repository’s own experiment script, using fixed seats and 100 hands, produced 98% and 54% respectively, consistent with these values.

Over 1600 decision states sampled evenly across the four phases at $d = 4$, the agent chose *raise* 1340 times, *call* 260 times and *fold* zero times, with raise fractions of 0.69, 0.83, 0.83 and 1.00 from preflop to river. The bluffing override

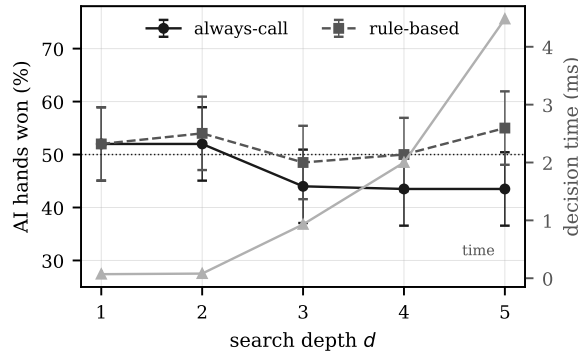


Fig. 4. Effect of the depth limit on outcome and cost. Win rates are shown with 95% binomial confidence intervals against the left axis and mean decision time against the right axis. Cost grows by roughly two orders of magnitude while win rate remains flat.

TABLE V
CONVERGENCE AND COST OF THE WIN-PROBABILITY ESTIMATOR

N	Mean $\hat{P}_{\text{win}}$	Within-state s.d.	Time (ms)
50	0.4973	0.0435	5.11
100	0.5039	0.0311	10.17
200	0.5057	0.0233	20.10
500	0.5070	0.0147	53.52
1000	0.5093	0.0096	102.73

cannot compensate, since it requires a pot of at least 100 chips: at the typical early value of 60 chips, none of 2000 sampled states triggered it; at 200 chips, 1932 of 2000 did.

Table IV and Fig. 4 address whether deeper search improves play. Against always-call the win rate falls from 52.0% at $d = 1$ to 43.5% at $d = 5$; that 8.5-point difference is about 1.7 standard errors of the difference of two proportions and is not significant at the 5% level, so the correct reading is that depth has no measurable effect rather than that deeper search is harmful. Against rule-based the rate fluctuates between 48.5% and 55.0% with no trend, while mean decision time rises from 0.07 ms to 4.48 ms, a factor of about 64.

D. Estimator Quality and Evaluator Agreement

Table V characterises the showdown simulator. The standard deviation of repeated estimates of the same state falls from 0.0435 at $N = 50$ to 0.0096 at $N = 1000$, closely tracking the $N^{-1/2}$ rate expected of an independent-sample estimator, while cost grows linearly. The interface uses $N = 250$, corresponding by interpolation to a standard deviation near two percentage points at about 25 ms.

Finally, the two hand evaluators were compared on 20 000 random seven-card sets. They agreed on the hand category in 44.27% of cases; in the remaining 55.73% the AI-side evaluator returned a strictly lower category, because it scores the first five known cards rather than the best five of seven.

VIII. DISCUSSION

The measurements locate the agent’s behaviour in its evaluation function rather than in its search, and the three principal results are mutually explanatory.

The zero fold count across 1600 states is the empirical face of Proposition 1 and the key to reading the match data. Since folding is dominated, the effective action set is {raise, call} and the agent’s policy is a never-fold, usually-raise policy. This immediately explains the asymmetry in Table III: an opponent that folds at random surrenders pots to a player who never does, producing a 99.5% win rate that reflects the opponent’s behaviour rather than the agent’s skill, while opponents that always contest the pot reduce the margin to parity. A high win rate against a weak baseline is therefore not evidence that the search is contributing.

The flatness of win rate in depth follows from the same source, together with two further mechanisms. First, because the action set is effectively binary, additional search can only reorder two alternatives. Second, values at different depths are not commensurable: leaves are scored by $U_{\text{heur}} \in [5, 100]$ whereas terminal folds are scored $\pm \Pi$, which is unbounded in chip units and grows as the hand progresses, so increasing d mixes the two scales at different rates without adding information about the outcome of the hand. Third, averaging at chance nodes compresses differences between sibling actions: because U_{heur} sees the board only through the coarse texture term $b(s)$ and the hand only through the first five known cards, the sampled successors of a chance node take similar values whose mean lies close to the parent’s own evaluation. The identical results at $d = 4$ and $d = 5$ against always-call, where the two searches select the same action in every hand of the seeded match, are consistent with this account.

The evaluator disagreement compounds the problem. Because $h(s)$ contributes up to 40 of the 100 available points in (15), and because the AI-side evaluator is strictly pessimistic in 55.7% of positions, the dominant term of the leaf value systematically understates the strength the engine will award at showdown. This is a correctable defect rather than a limitation of the method.

The Monte Carlo agent’s poor showing has a separate and equally specific cause. Its policy raises only when the simulated mean hand category reaches 5, corresponding to a flush or better, and folds whenever the mean falls below 2 while facing a bet. Since the mean category of a completed random board typically lies between 0 and 2, the agent folds in most contested spots. The sampling itself is sound—the win-probability estimator in Table V converges at the expected rate—so this is a threshold-calibration failure, not a defect in the simulation. It is instructive that the system already computes a well-behaved probability estimate for display but drives its decisions from a coarse category average instead, which is precisely the substitution that simulation-based poker agents avoid [6].

Set against the literature the pattern is unsurprising: heuristic evaluation over a sampled stochastic tree optimises a hand-crafted proxy rather than expected value in chips, and the shift towards regret-based equilibrium approximation [8], [9] and learned rather than hand-tuned evaluation [10] was motivated in part by this class of failure [5].

IX. LIMITATIONS AND FUTURE WORK

The evaluation function is the binding constraint on decision quality: its positive lower bound makes folding unreachable, its hand-strength term comes from the weaker of the two evaluators, and its output is not expressed in the same units as the terminal fold payoff. The chance-node approximation is unbiased but shallow—one card is sampled per node, the sample size is not adapted to the node’s depth, and no variance-reduction scheme is applied. The opponent model is a substitution rather than a model: MIN nodes minimise the agent’s own utility computed from the agent’s own cards, and the search samples from a deck that excludes the opponent’s real hole cards, a small information leak in the agent’s favour. The betting abstraction is coarse, with fixed raise sizes and with all-in dynamics invisible to the search, and the bluffing rule is a post-search override whose pot threshold makes it inert early and near-deterministic late. Finally, at 200 hands per cell differences below roughly seven percentage points are indistinguishable, and because stacks are reset each hand the protocol measures per-hand outcome rather than long-run bankroll performance.

The measurements imply a clear ordering of improvements. The most immediate is to express leaf and terminal values on a common expected-chip scale, which would restore folding as a reachable action and make values comparable across depths. Consolidating the two hand evaluators onto the engine’s best-of-seven implementation would remove the pessimism in $h(s)$ at no algorithmic cost, and re-calibrating the Monte Carlo thresholds against measured win probability rather than raw hand category would address the weakness in Table III. Beyond repair, the heuristic strength term could be replaced by the win-probability estimate the system already computes, and `OpponentProfile` could be connected so that MIN nodes reflect observed action frequencies rather than worst-case play, moving the agent toward the opponent-modelling approach of Southey *et al.* [7]. Introducing ϵ -minimax cutoffs at chance nodes [2] would make a larger branching factor affordable. Further out, the appropriate successors to a sampled chance node are Monte Carlo tree search with a principled selection rule [13], [14] and counterfactual regret minimisation [8], [9]. None of these is implemented in the present system; they are stated as directions, not as results.

Several quantities were not measured and should not be inferred from this paper: win rate against human opponents, long-run bankroll performance, tournament placement statistics across many tournaments, performance against any external poker agent, and the effect of disabling the bluffing rule. Each requires an experiment that has not been conducted.

X. CONCLUSION

This paper presented a Texas Hold’em system whose artificial player uses depth-limited Expectiminimax search with uniformly sampled chance nodes, and reported an empirical analysis of that agent. The engine implements the game correctly—best-of-seven evaluation, side pots, tournaments and a graphical interface—and the search implements the

MAX, MIN, chance and leaf backup rules faithfully, with cost scaling as theory predicts: node count jumps by a factor of twenty at the depth where the first chance node becomes reachable, and grows linearly in the chance branching factor.

The measurements, however, place the agent’s behaviour in its evaluation function rather than its search. Because the leaf value is bounded below by a positive constant while a fold is scored as the negative of the pot, folding is provably dominated and was selected in none of 1600 sampled states. That result explains both the 99.5% win rate against an opponent that folds at random and the near-parity outcomes against opponents that do not, and it explains why raising the depth limit from one to five plies multiplied decision cost by roughly sixty-four while leaving the win rate statistically unchanged. The system’s two hand evaluators were separately found to agree on only 44.3% of seven-card sets. Each improvement identified above is traceable to a specific measurement reported here.

DISCLOSURE OF AI TOOL USAGE

Generative AI tools were used during this project and their role is disclosed in accordance with course requirements: (i) code review and debugging support during implementation, (ii) static analysis of the completed source tree to recover its structure and identify the properties reported in Sections V-E and VII, (iii) construction of the measurement harness described in Section VII, and (iv) drafting and language editing of this manuscript. All quantitative results were produced by executing the project’s own code; no figure, table or numerical value was generated by a language model. System design, algorithm selection and implementation decisions are the authors’ own, and every technical claim has been verified against the source code and the recorded measurements.

REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.
- [2] B. W. Ballard, “The ϵ -minimax search procedure for trees containing chance nodes,” *Artif. Intell.*, vol. 21, no. 3, pp. 327–350, 1983.
- [3] D. Koller and A. Pfeffer, “Representations and solutions for game-theoretic problems,” *Artif. Intell.*, vol. 94, no. 1–2, pp. 167–215, 1997.
- [4] D. Billings, A. Davidson, J. Schaeffer, and D. Szafron, “The challenge of poker,” *Artif. Intell.*, vol. 134, no. 1–2, pp. 201–240, 2002.
- [5] J. Rubin and I. Watson, “Computer poker: A review,” *Artif. Intell.*, vol. 175, no. 5–6, pp. 958–987, 2011.
- [6] D. Billings, L. Peña, J. Schaeffer, and D. Szafron, “Using probabilistic knowledge and simulation to play poker,” in *Proc. 16th Nat. Conf. Artif. Intell. (AAAI)*, 1999, pp. 697–703.
- [7] F. Southey, M. Bowling, B. Larson, C. Piccione, N. Burch, D. Billings, and C. Rayner, “Bayes’ bluff: Opponent modelling in poker,” in *Proc. 21st Conf. Uncertainty Artif. Intell. (UAI)*, 2005, pp. 550–558.
- [8] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, “Regret minimization in games with incomplete information,” in *Advances in Neural Inf. Process. Syst. (NeurIPS)* 20, 2007, pp. 1729–1736.
- [9] M. Bowling, N. Burch, M. Johanson, and O. Tammelin, “Heads-up limit hold’em poker is solved,” *Science*, vol. 347, no. 6218, pp. 145–149, 2015.
- [10] M. Moravčík *et al.*, “DeepStack: Expert-level artificial intelligence in heads-up no-limit poker,” *Science*, vol. 356, no. 6337, pp. 508–513, 2017.
- [11] N. Brown and T. Sandholm, “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals,” *Science*, vol. 359, no. 6374, pp. 418–424, 2018.

- [12] N. Brown and T. Sandholm, "Superhuman AI for multiplayer poker," *Science*, vol. 365, no. 6456, pp. 885–890, 2019.
- [13] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proc. 17th Eur. Conf. Mach. Learn. (ECML)*, 2006, pp. 282–293.
- [14] C. B. Browne *et al.*, "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, 2012.